

Research Note — LLD Practice Platform

1. Learner Problem

Low-Level Design (LLD) is a core competency evaluated in software engineering interviews. Candidates are expected to design class hierarchies, define responsibilities, model relationships, and justify their decisions under constraints — skills that require deliberate practice to develop.

The gap between understanding LLD theory and being able to produce a good design is wide:

- **Reading is not enough.** A learner can understand SOLID principles or design patterns in the abstract but struggle to apply them to a concrete, open-ended scenario such as designing a Parking Lot or a Splitwise-style expense splitter.
- **Self-evaluation is unreliable.** Multiple valid designs exist for the same problem. Without an external benchmark, learners cannot judge whether their class responsibilities are well-defined, their interfaces are appropriate, or their abstractions reduce coupling.
- **No iteration loop.** Most practice happens in isolation — a whiteboard or a text file — with no mechanism to track improvement across attempts or understand *why* one design is better than another.

What learners need is a structured, repeatable practice loop: submit a design, receive specific rubric-based feedback, understand the reasoning, and try again.

2. Existing Approaches and Tools

Coding/LLD interview platforms provide curated problems and reference solutions. They are useful for exposure to a variety of scenarios, but feedback is typically limited to comparing the learner's output against one canonical solution. Design is inherently open-ended; a single reference answer does not capture all valid approaches.

Documentation and tutorial articles explain design patterns and walk through worked examples. They are a good starting point for theory but are one-directional — there is no mechanism for a learner to submit their own design and receive feedback on it.

Generic online coding judges (competitive programming platforms) evaluate code correctness through test cases. They are well-suited for algorithmic problems but are not designed to assess object-oriented design quality: encapsulation, interface design, coupling, extensibility, and design rationale are outside the scope of test-case execution.

AI assistants and chat tools can provide on-demand feedback on a design described in natural language. They are flexible and often insightful, but they do not provide a structured practice workflow

— there is no problem library, no consistent rubric, no submission history, and no improvement tracking across attempts.

3. Gaps

Taken together, the existing approaches leave specific gaps for LLD practice:

Gap	Why it matters
Generic judges test execution, not design	A working program can still have poor encapsulation, high coupling, or missing abstractions
Reference solutions show one valid design	Feedback should address <i>the learner's</i> design, not penalise valid alternatives
AI chat lacks a structured practice workflow	No rubric consistency, no history, no retry loop
No per-criterion explainability	Learners need to know <i>which</i> aspect of their design is weak and <i>why</i>
Multiple valid designs make answer comparison unsuitable	Evaluation must assess design principles, not pattern-match against a fixed answer

4. Product Direction

The LLD Practice Platform addresses these gaps with a focused, rubric-based design practice tool.

Core workflow: Learner selects a problem → receives full requirements and assumptions → submits a structured design → receives per-criterion rubric feedback → can review, retry, and track improvement.

Structured submission format: Rather than free-form text or code files, learners provide named fields: assumptions, class definitions (with responsibilities and methods), interfaces, relationships, design explanation, and edge cases. This structured input makes evaluation consistent and gives both deterministic and AI-based evaluators a common representation to work against.

Dual evaluation strategy: - *RuleBasedEvaluator* applies deterministic heuristics: class count, interface presence, keyword coverage, section completeness, design explanation depth. It produces consistent, explainable scores without depending on an external API. - *AIEvaluator* sends the structured submission to an OpenAI model with an explicit rubric prompt, requesting evidence-based, criterion-level feedback. It handles the subjective reasoning aspects that heuristics cannot fully capture.

If the AI evaluator fails at runtime for any reason (timeout, API error, malformed response, or validation failure), the platform automatically falls back to the *RuleBasedEvaluator* and marks the result accordingly, so the learner always receives usable feedback.

Extensibility: The evaluator abstraction (`BaseEvaluator`) allows additional evaluator types — human review, code execution, diagram analysis — to be added without changing the service or API layer.

Attempt history and retry: All submissions are persisted before evaluation begins. Learners can view all past attempts, track score improvement across retries, and re-evaluate any failed evaluation via a dedicated endpoint.

5. Key Design Insight

LLD quality cannot be captured by a single score. A design may demonstrate strong requirement coverage while exhibiting poor coupling, or show elegant abstraction while failing to handle edge cases. Meaningful feedback requires evaluating distinct, independently scorable dimensions.

The platform uses an 8-criterion rubric, chosen to map to the aspects most commonly assessed in LLD interviews and most instructive for learners:

Criterion	What it captures
Requirement Understanding	Does the design address the stated functional requirements?
Class Responsibilities	Are responsibilities well-defined and singular?
Encapsulation & Interfaces	Are abstractions and contracts clearly expressed?
Coupling & Cohesion	Are dependencies minimal and directed appropriately?
Abstraction / Design Patterns	Are recognised patterns applied where they add value?
Extensibility	Can the design accommodate new requirements without major rework?
Edge Cases & Testability	Are failure modes considered? Is the design easy to test?
Design Explanation	Does the learner articulate rationale and trade-offs?

Scoring each criterion independently, with evidence drawn from the learner's own submission, produces feedback that is specific, actionable, and fair across the variety of valid designs that any open-ended LLD problem admits.